Quad Tree

Last Updated : 24 Feb, 2025

Quadtrees are trees used to efficiently store data of points on a two-dimensional space. Each node of a Quad Tree has at most four children. We can construct a quadtree from a two-dimensional area using the following steps:

1. Divide the current two-dimensional space into four boxes.
2. If a box contains one or more points in it, create a child object, storing in it the two-dimensional space of the box.
3. If a box does not contain any points, do not create a child for it.
4. Recurse for each of the children.

Quadtrees are used in image compression, where each node contains the average colour of each of its children. The deeper you traverse in the tree, the more the detail of the image. Quadtrees are also used in searching for nodes in a two-dimensional area. For instance, if you want to find the closest point to given coordinates, you can do it using quadtrees.

Insert Function:

The insert functions is used to insert a node into an existing Quad Tree. This function first checks whether the given node is within the boundaries of the current quad. If it is not, then we immediately cease the insertion. If it is within the boundaries, we select the appropriate child to contain this node based on its location. This function is O(Log N) where N is the size of distance.

Search Function:

The search function is used to locate a node in the given quad. It can also be modified to return the closest node to the given point.

This function is implemented by taking the given point, comparing with the boundaries of the child quads and recursing. This function is O(Log N) where N is size of distance.

The program given below demonstrates storage of nodes in a quadtree:

CPP Java Python C# JavaScript

```cpp
// C++ Implementation of Quad Tree
#include <cmath>
#include <iostream>
using namespace std;

// Used to hold details of a point
struct Point {
    int x;
    int y;
    Point(int _x, int _y)
    {
        x = _x;
        y = _y;
    }
    Point()
    {
        x = 0;
        y = 0;
    }
};

// The objects that we want stored in the quadtree
struct Node {
    Point pos;
    int data;
    Node(Point _pos, int _data)
    {
        pos = _pos;
        data = _data;
    }
    Node() { data = 0; }
};
```

```
// The main quadtree class
class Quad {
    // Hold details of the boundary of this node
    Point topLeft;
    Point botRight;

    // Contains details of node
    Node* n;

    // Children of this tree
    Quad* topLeftTree;
    Quad* topRightTree;
    Quad* botLeftTree;
    Quad* botRightTree;

public:
    Quad()
    {
        topLeft = Point(0, 0);
        botRight = Point(0, 0);
        n = NULL;
        topLeftTree = NULL;
        topRightTree = NULL;
        botLeftTree = NULL;
        botRightTree = NULL;
    }
    Quad(Point topL, Point botR)
    {
        n = NULL;
        topLeftTree = NULL;
        topRightTree = NULL;
        botLeftTree = NULL;
        botRightTree = NULL;
        topLeft = topL;
        botRight = botR;
    }
    void insert(Node*);
    Node* search(Point);
    bool inBoundary(Point);
```

```
};

// Insert a node into the quadtree
void Quad::insert(Node* node)
{
    if (node == NULL)
        return;

    // Current quad cannot contain it
    if (!inBoundary(node->pos))
        return;

    // We are at a quad of unit area
    // We cannot subdivide this quad further
    if (abs(topLeft.x - botRight.x) <= 1
        && abs(topLeft.y - botRight.y) <= 1) {
        if (n == NULL)
            n = node;
        return;
    }

    if ((topLeft.x + botRight.x) / 2 >= node->pos.x) {
        // Indicates topLeftTree
        if ((topLeft.y + botRight.y) / 2 >= node->pos.y) {
            if (topLeftTree == NULL)
                topLeftTree = new Quad(
                    Point(topLeft.x, topLeft.y),
                    Point((topLeft.x + botRight.x) / 2,
                          (topLeft.y + botRight.y) / 2));
            topLeftTree->insert(node);
        }

        // Indicates botLeftTree
        else {
            if (botLeftTree == NULL)
                botLeftTree = new Quad(
                    Point(topLeft.x,
                          (topLeft.y + botRight.y) / 2),
                    Point((topLeft.x + botRight.x) / 2,
                          botRight.y));
```

```
            botLeftTree->insert(node);
        }
    }
    else {
        // Indicates topRightTree
        if ((topLeft.y + botRight.y) / 2 >= node->pos.y) {
            if (topRightTree == NULL)
                topRightTree = new Quad(
                    Point((topLeft.x + botRight.x) / 2,
                          topLeft.y),
                    Point(botRight.x,
                          (topLeft.y + botRight.y) / 2));
            topRightTree->insert(node);
        }

        // Indicates botRightTree
        else {
            if (botRightTree == NULL)
                botRightTree = new Quad(
                    Point((topLeft.x + botRight.x) / 2,
                          (topLeft.y + botRight.y) / 2),
                    Point(botRight.x, botRight.y));
            botRightTree->insert(node);
        }
    }
}

// Find a node in a quadtree
Node* Quad::search(Point p)
{
    // Current quad cannot contain it
    if (!inBoundary(p))
        return NULL;

    // We are at a quad of unit length
    // We cannot subdivide this quad further
    if (n != NULL)
        return n;

    if ((topLeft.x + botRight.x) / 2 >= p.x) {
```

```
        // Indicates topLeftTree
        if ((topLeft.y + botRight.y) / 2 >= p.y) {
            if (topLeftTree == NULL)
                return NULL;
            return topLeftTree->search(p);
        }

        // Indicates botLeftTree
        else {
            if (botLeftTree == NULL)
                return NULL;
            return botLeftTree->search(p);
        }
    }
    else {
        // Indicates topRightTree
        if ((topLeft.y + botRight.y) / 2 >= p.y) {
            if (topRightTree == NULL)
                return NULL;
            return topRightTree->search(p);
        }

        // Indicates botRightTree
        else {
            if (botRightTree == NULL)
                return NULL;
            return botRightTree->search(p);
        }
    }
};

// Check if current quadtree contains the point
bool Quad::inBoundary(Point p)
{
    return (p.x >= topLeft.x && p.x <= botRight.x
            && p.y >= topLeft.y && p.y <= botRight.y);
}

// Driver program
int main()
```

```
{
    Quad center(Point(0, 0), Point(8, 8));
    Node a(Point(1, 1), 1);
    Node b(Point(2, 5), 2);
    Node c(Point(7, 6), 3);
    center.insert(&a);
    center.insert(&b);
    center.insert(&c);
    cout << "Node a: " << center.search(Point(1, 1))->data
         << "\n";
    cout << "Node b: " << center.search(Point(2, 5))->data
         << "\n";
    cout << "Node c: " << center.search(Point(7, 6))->data
         << "\n";
    cout << "Non-existing node: "
         << center.search(Point(5, 5));
    return 0;
}
```

Output

```
Node a: 1
Node b: 2
Node c: 3
Non-existing node: 0
```

Exercise: Implement a Quad Tree which returns 4 closest nodes to a given point.

Comment

A Aditya Kamath 1

19

Article Tags : Tree Divide and Conquer Searching Advanced Data Structure

+1 More

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play

Download on the App Store

Company

About Us

Legal

Privacy Policy

Contact Us

Advertise with us

GFG Corporate Solution

Campus Training Program

Explore

POTD

Job-A-Thon

Blogs

Nation Skill Up

Tutorials

Courses

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved